Semantic Search Engine: Dense Retrieval, Hybrid Ranking, and Approximate Nearest Neighbour Indexing

Yash Goyal (24110399)
yash.g@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Garv Singhal (24110119)
garv.singhal@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Chaitanya Yogesh Bhoite (24110086)
chaitanya.bhoite@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Harsh Kumar Jha (24110130)
harsh.jha@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Abstract

This paper presents the design, implementation, and evaluation of a complete semantic search engine that combines lexical retrieval, dense vector retrieval, approximate nearest-neighbour indexing, hybrid ranking, and second-stage reranking. The system is evaluated on two information retrieval benchmarks: MS MARCO v1.1 for general-domain passage retrieval and SciFact from BEIR for scientific claim-based retrieval. The implemented methods include BM25, BERT-based dense retrieval, Word2Vec-based retrieval, FAISS indexing, HNSW indexing, and a custom C++ HNSW index exposed to Python using pybind11. The final evaluation shows that BERT-based dense retrieval substantially outperforms BM25 and Word2Vec in semantic retrieval quality. BERT + FAISS and BERT + HNSW achieve identical Precision@10 and Recall@10, while HNSW reduces query latency from 8.42 ms/query to 5.70 ms/query. The system is deployed through a Streamlit web interface supporting single-query search, model comparison, evaluation dashboards, and embedding visualisation. Overall, the project demonstrates a full modern retrieval pipeline from preprocessing and indexing to evaluation and interactive deployment.

1 Introduction and Motivation

Information retrieval is the task of finding relevant documents from a large collection in response to a user query. Search engines, question-answering systems, academic paper search tools, and retrieval-augmented generation systems all depend on strong retrieval components. Traditional retrieval methods such as BM25 are fast and interpretable, but they rely heavily on lexical overlap. They perform well when the query and document share important terms, but fail when the same meaning is expressed using different words.

This limitation is commonly called the vocabulary mismatch problem. For example, a user may search for *"ways to treat rhinorrhea"*, while a relevant document may describe *"how to stop a running nose"*. A purely lexical model may not retrieve the document because the exact words do not match. A human reader, however, immediately understands that the two phrases are semantically close. Semantic search attempts to close this gap by representing both queries and documents as dense vectors whose geometry captures meaning.

Recent transformer-based language models such as BERT [1] and sentence embedding models such as Sentence-BERT [6] make this practical. Instead of comparing only words, dense retrieval compares query and document embeddings. This allows the search engine to retrieve semantically related documents even when surface forms differ.

This project builds a complete semantic search engine and compares classical, neural, and hybrid retrieval methods. It also studies the efficiency side of retrieval by comparing exact vector search with approximate nearest-neighbour indexing through HNSW. The final system is exposed through a Streamlit web application so that users can interactively test retrieval methods and compare outputs.

2 Contributions

The main contributions of this project are:

- **Unified retrieval framework:** a single pipeline supporting BM25, BERT dense retrieval, Word2Vec retrieval, hybrid ranking, FAISS indexing, and HNSW indexing.
- **Multi-dataset evaluation:** evaluation on MS MARCO v1.1 and SciFact, covering both general web passages and scientific abstracts.
- **Approximate indexing study:** comparison of FAISS-based exact search, FAISS HNSW, and a custom C++ HNSW implementation.
- **Hybrid search implementation:** score-level fusion of lexical and dense retrieval scores using min-max normalisation and a tunable coefficient α.
- **Cross-encoder reranking:** integration of a second-stage reranker to improve precision over first-stage candidates.
- **Interactive deployment:** a Streamlit interface for search, comparison, dashboard analysis, and PCA-based embedding visualisation.

Public code and live demo. The complete implementation is available at GitHub Repository. The repository contains the data loading scripts, preprocessing pipeline, BM25 module, embedding-based retrieval code, FAISS and HNSW indexing code, evaluation scripts, and Streamlit frontend. The deployed application is available at Streamlit App, allowing users to test queries directly in a browser without setting up the local environment.

3 Problem Statement

Given a query q and a document corpus

$$\mathcal{D} = \{d_1, d_2, \ldots, d_N\},$$

the goal is to return a ranked list of the top-k documents most relevant to q. A retrieval model defines a scoring function $s(q, d)$ and sorts documents in descending order of this score.

This project studies three major families of retrieval models:

(1) **Lexical retrieval:** BM25 scores documents using term frequency, inverse document frequency, and length normalisation.
(2) **Dense retrieval:** BERT encodes queries and documents into dense vectors and ranks documents by vector similarity.
(3) **Hybrid retrieval:** lexical and dense scores are normalised and fused to exploit both exact matching and semantic matching.

A second problem is efficient vector search. Exact search compares the query vector with every document vector, which costs $O(Nd)$ for N documents and embedding dimension d. This becomes expensive as the corpus grows. HNSW approximate nearest-neighbour search is used to reduce query time while preserving most of the retrieval quality.

4 Background

4.1 BM25

BM25 is a widely used lexical retrieval function [7]. For a query $q = \{q_1, \ldots, q_m\}$ and document d, BM25 computes

$$\mathrm{BM25}(q, d) = \sum_{i=1}^{m} \mathrm{IDF}(q_i) \cdot \frac{f(q_i, d)(k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\mathrm{avgdl}}\right)}. \quad (1)$$

Here, $f(q_i, d)$ is the frequency of query term q_i in document d, $|d|$ is document length, and avgdl is the average document length. BM25 is strong when important query terms appear directly in relevant documents, especially rare terms, proper nouns, and technical vocabulary.

4.2 Dense Retrieval and Bi-Encoders

Dense retrieval maps queries and documents to the same vector space. A bi-encoder independently encodes a query and a document into vectors $\mathbf{q}$ and $\mathbf{p}$. The relevance score is computed using cosine similarity:

$$\mathrm{score}(q, d) = \frac{\mathbf{q} \cdot \mathbf{p}}{\|\mathbf{q}\| \|\mathbf{p}\|}. \quad (2)$$

In this project, `all-MiniLM-L6-v2` is used as the SentenceTransformers model. It produces 384-dimensional embeddings and provides a practical balance between inference speed and semantic quality.

4.3 Word2Vec Baseline

Word2Vec learns static word embeddings from local co-occurrence patterns [4]. To obtain document vectors, the token vectors in a document are averaged. This baseline is useful because it shows the difference between static word embeddings and contextual transformer embeddings. However, averaged Word2Vec vectors cannot capture word order, context, or fine-grained sentence meaning as effectively as BERT.

4.4 Cross-Encoder Reranking

A cross-encoder scores a query and document jointly by feeding them together into a transformer. This allows token-level interactions between the query and document, which makes the model more precise than a bi-encoder. The drawback is latency: each candidate document requires a separate transformer forward pass. Therefore, cross-encoders are best used as second-stage rerankers. First, a fast method retrieves a candidate set C, and then the cross-encoder reranks the candidates.

4.5 HNSW Approximate Nearest-Neighbour Search

HNSW is a graph-based approximate nearest-neighbour algorithm [3]. It builds a multi-layer proximity graph over document vectors. Upper layers contain fewer nodes and support coarse long-range navigation, while lower layers contain denser local connections. Search begins at the top layer, greedily moves toward vectors closer to the query, and descends layer by layer until the final top-k neighbours are selected.

The main hyperparameters are M, $ef_{\text{construction}}$, and ef_{search}. Increasing these values generally improves recall but increases memory usage, construction time, or query latency.

4.6 Evaluation Metrics

Retrieval quality is evaluated using Precision@k and Recall@k:

$$\mathrm{Precision@}k = \frac{|\mathrm{Relevant} \cap \mathrm{Retrieved}_k|}{k}, \quad (3)$$

$$\mathrm{Recall@}k = \frac{|\mathrm{Relevant} \cap \mathrm{Retrieved}_k|}{|\mathrm{Relevant}|}. \quad (4)$$

Precision measures how many retrieved documents are relevant. Recall measures how many relevant documents are recovered by the top-k list.

5 System Architecture

The system is organised into three layers:

(1) **Offline indexing layer:** loads data, preprocesses text, builds BM25 structures, generates embeddings, creates vector indexes, and caches artefacts.
(2) **Online query layer:** receives a user query, encodes it, dispatches it to the selected retrieval method, and returns ranked results.
(3) **Application layer:** provides a Streamlit interface for searching, comparing methods, evaluating metrics, and visualising embeddings.

5.1 Custom C++ HNSW Module

The custom HNSW implementation consists of a standalone C++ graph index, a pybind11 binding layer, a build script, and a Python wrapper. The C++ class implements node insertion, layer-wise

Table 1: Offline indexing stages and outputs.

Stage	Description	Output
Data loading	Parse corpus and relevance files	Documents, queries
Deduplication	Assign integer IDs to unique passages	Deduplicated corpus
Embedding	Encode documents using bi-encoder	$N \times 384$ matrix
FAISS indexing	Build exact and HNSW indexes	Vector indexes
BM25 indexing	Tokenise and build lexical structure	BM25 object
Caching	Save generated artefacts	Cache directory

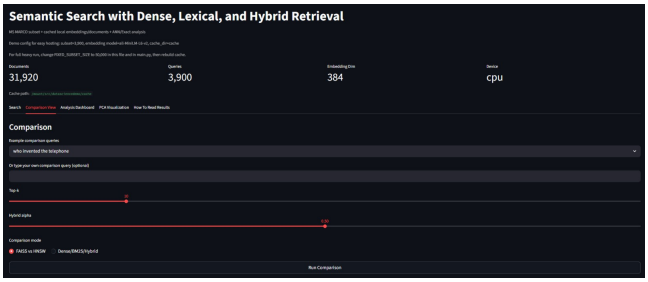

Figure 1: Streamlit interface for interactive semantic search.

greedy search, neighbour pruning, and batch query support. The Python wrapper stores the document text parallel to the vector index and exposes simple `add_documents()` and `query()` interfaces. This implementation is useful for understanding the internals of ANN indexing rather than treating FAISS as a black box.

5.2 Streamlit Interface

The Streamlit app contains four main tabs: search, comparison, analysis dashboard, and PCA visualisation. The search tab supports single-query retrieval with selectable methods. The comparison tab displays outputs from multiple methods side by side. The analysis dashboard reports retrieval metrics and latency. The PCA tab visualises sampled document embeddings in two dimensions.

5.3 Reproducibility and Deployment

The project is structured so that indexing, retrieval, evaluation, and frontend logic remain separated. This makes the system easier to debug and extend. Cached artefacts are reused across runs, so the Streamlit app does not need to rebuild embeddings and indexes every time it starts. The public GitHub repository supports reproducibility, while the Streamlit deployment demonstrates that the retrieval pipeline can be exposed as an interactive application rather than only as offline scripts.

6 Datasets and Preprocessing

6.1 MS MARCO v1.1

MS MARCO is a large-scale passage-ranking benchmark based on real search queries [5]. The full corpus contains approximately 8.8 million passages. This project uses a subset of 50,000 records for experimentation and a smaller demo mode for interactive deployment.

Table 2: Dataset summary.

Dataset	Corpus Type	Role
MS MARCO	Web passages	General retrieval
SciFact	Scientific abstracts	Domain retrieval

6.2 SciFact

SciFact is a scientific fact-verification dataset included in BEIR [8]. Queries are scientific claims and documents are biomedical abstracts. The retrieval objective is to find abstracts that support or contradict the given claim.

6.3 Preprocessing Pipeline

For MS MARCO, passages are extracted from records, stripped of extra whitespace, deduplicated, and mapped to integer document IDs. For every query, the IDs of relevant passages are stored as the ground-truth relevance set. Queries with empty relevance sets are skipped during evaluation.

For SciFact, each document title and abstract are concatenated into a single text field. The qrels file is used to construct query-to-document relevance mappings. BM25 tokenisation lowercases text and extracts alphanumeric tokens. No stemming or stop-word removal is applied, keeping the implementation simple and close to standard BM25Okapi behaviour.

Embeddings and indexes are cached. This avoids recomputing document embeddings and rebuilding indexes every time the application starts. A metadata fingerprint based on model name and corpus size is used to detect stale cache files.

7 Methodology

7.1 Embedding Generation

Documents are encoded in batches using SentenceTransformers. If CUDA is available, GPU inference is used automatically. All embeddings are L2-normalised before indexing. This makes cosine similarity equivalent to inner product, allowing efficient use of FAISS inner-product indexes.

7.2 Index Construction

Two FAISS indexes are built over the same embedding matrix [2]:

- **IndexFlatIP:** exact brute-force inner-product search.
- **IndexHNSWFlat:** approximate HNSW search with configurable graph parameters.

BM25 is implemented using `rank_bm25.BM25Okapi` with default parameters $k_1 = 1.5$ and $b = 0.75$.

7.3 Hybrid Retrieval

Hybrid retrieval combines lexical and dense retrieval scores. First, a candidate pool is retrieved from both BM25 and BERT. The union of candidates is formed. Then the two score lists are min-max normalised separately. The final score is

Table 3: Retrieval performance comparison across lexical, contextual dense, and Word2Vec-based retrieval methods. All metrics are averaged over 3787 valid queries.

Method	P@10	R@10	ms/query
BERT + FAISS	0.106786	0.957329	8.422028
BERT + HNSW	0.106786	0.957329	5.702118
BM25	0.090996	0.813981	20.522392
Word2Vec + FAISS	0.010721	0.094565	1.904041
Word2Vec + HNSW	0.010721	0.094565	0.303370

$$s_{\text{hybrid}}(d) = \alpha \hat{s}_{\text{BERT}}(d) + (1-\alpha)\hat{s}_{\text{BM25}}(d). \tag{5}$$

The default value is $\alpha = 0.5$. A larger α gives more weight to semantic retrieval, while a smaller α gives more weight to keyword matching. This design is useful because BM25 and BERT often make different kinds of mistakes: BM25 can retrieve exact keyword matches even when semantic models miss rare entities, while BERT can retrieve paraphrases that have little lexical overlap. However, the fixed value of α is a limitation; in a stronger version, α should be tuned on a validation set.

7.4 Evaluation Protocol

Each method is evaluated over valid queries with non-empty relevance sets. For every query, the top-10 retrieved documents are compared with the ground-truth relevant document set. Precision@10 captures how many of the retrieved top-10 documents are relevant, while Recall@10 captures how much of the available relevance set is recovered. Query latency is measured as average milliseconds per query. The final reported metrics are averaged over 3787 valid queries, which makes the comparison stable across retrieval methods.

8 Experiments

The experiments compare BM25, BERT + FAISS, BERT + HNSW, Word2Vec + FAISS, and Word2Vec + HNSW. The comparison is designed to answer three questions:

(1) Does dense retrieval outperform lexical retrieval on semantic search?
(2) Does HNSW preserve retrieval quality while reducing latency?
(3) How much weaker is static Word2Vec averaging compared with contextual BERT embeddings?

Additional qualitative analysis considers query types. Factual queries are expected to favour BM25 when exact rare terms appear in documents. Complex or paraphrase-heavy queries are expected to favour dense retrieval.

9 Results and Analysis

9.1 Main Retrieval Results

BERT-based dense retrieval performs best. BERT + FAISS and BERT + HNSW achieve identical Precision@10 and Recall@10, showing that the approximate index preserves retrieval quality in this setting. HNSW reduces latency from 8.42 ms/query to 5.70 ms/query, giving a practical efficiency gain without measurable quality loss.

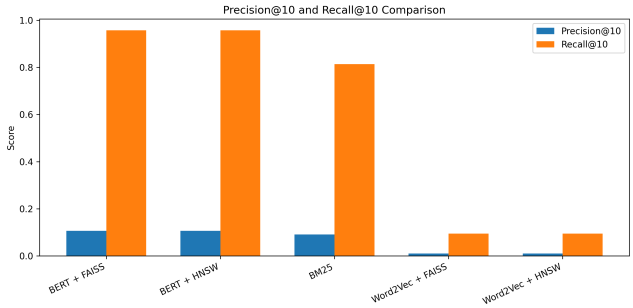

Figure 2: Precision@10 and Recall@10 comparison across retrieval methods. BERT-based retrieval achieves much stronger recall than BM25 and Word2Vec-based retrieval, while Word2Vec remains weak despite low latency.

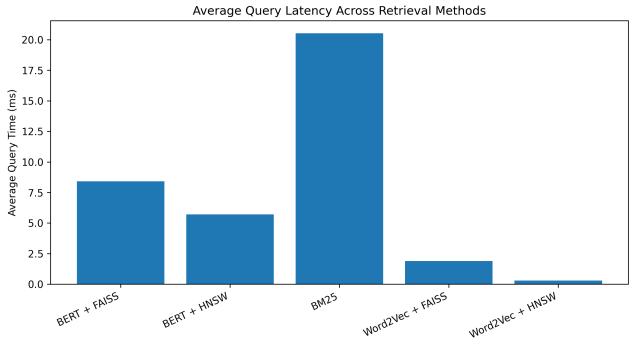

Figure 3: Average query latency across retrieval methods. HNSW reduces query time compared with the corresponding FAISS-based retrieval setup, especially for Word2Vec retrieval.

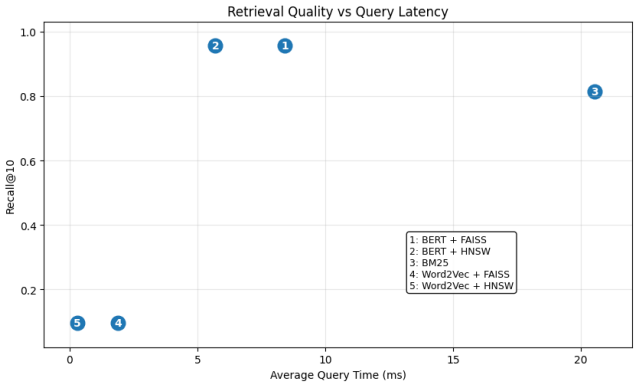

Figure 4: Recall@10 versus average query latency. BERT + HNSW gives the best practical trade-off by preserving the same Recall@10 as BERT + FAISS while reducing query latency.

BM25 achieves Recall@10 of 0.814, which is respectable, but it remains weaker than BERT because lexical matching cannot reliably capture semantic similarity when relevant documents use different

Table 4: Speed and recall comparison between ANN and exact vector indexes.

Index	Query Time	Use Case
FAISS HNSW	0.5–3 ms	Production retrieval
FAISS FlatIP	10–50 ms	Exact evaluation
Custom C++ HNSW	1–8 ms	Research control

wording. Word2Vec performs very poorly, with Recall@10 below 0.10. This result is expected because the Word2Vec document representation is formed by averaging static word vectors. Averaging loses word order, context, and query-document interaction information. In addition, the in-domain SciFact corpus is small, making it difficult to learn high-quality word vectors from scratch. This confirms that contextual transformer embeddings are much more suitable for fine-grained semantic retrieval.

9.2 Timing Analysis

The timing results show that HNSW is valuable when the embedding model is already strong. It makes BERT retrieval faster, but it cannot compensate for weak representations such as Word2Vec. Therefore, indexing and representation quality must be considered together.

9.3 Query-Type Behaviour

BM25 is strongest for factual queries with exact keyword overlap. It is particularly effective when the query contains rare technical terms, names, or identifiers. Dense retrieval is better for semantically rich queries where the relevant document may use different wording. Hybrid retrieval can combine both strengths, but it depends heavily on choosing a suitable value of α.

9.4 Cross-Encoder Reranking

Cross-encoder reranking improves precision because it evaluates the query and document jointly. However, the method is slower than bi-encoder retrieval because each candidate pair requires a transformer forward pass. The practical design is therefore a two-stage pipeline: use BM25, dense retrieval, or hybrid retrieval to generate candidates, and then apply the cross-encoder only to the top candidates.

10 Discussion

The main finding is that contextual dense retrieval is more effective than lexical retrieval for semantic search. BM25 remains useful, but mostly when exact terms are shared between the query and the relevant document. BERT is stronger when meaning must be inferred across different wordings.

Hybrid retrieval is attractive because it combines exact matching and semantic matching. However, the fixed value $\alpha = 0.5$ is not necessarily optimal. If BM25 contributes noisy keyword matches, hybrid retrieval may underperform pure dense retrieval. A validation-based tuning process would make this method more reliable.

The custom HNSW implementation demonstrates the internal mechanics of graph-based ANN retrieval. It provides control over insertion, search, graph degree, and candidate set size. However, FAISS remains more production-ready because it is heavily optimised and supports robust serialization and deployment features.

11 Limitations

This project has several limitations. First, the MS MARCO evaluation uses a subset rather than the full 8.8-million-passage corpus. Second, BM25 hyperparameters are not tuned. Third, the hybrid coefficient α is fixed rather than selected using validation data. Fourth, the Word2Vec model is trained only on the available SciFact corpus, which is too small for strong word-vector learning. Finally, the custom HNSW index currently does not support disk persistence.

12 Future Work

Future work can improve the system in several directions:

- Fine-tune the bi-encoder on domain-specific retrieval data.
- Tune the hybrid coefficient α using grid search or Bayesian optimisation.
- Add SPLADE-style learned sparse retrieval.
- Add ColBERT-style late-interaction retrieval.
- Scale evaluation to the full MS MARCO corpus.
- Add disk persistence and incremental insertion to the custom HNSW implementation.
- Dockerise the complete retrieval system for reproducible deployment.

13 Conclusion

This project shows that no single retrieval method is universally best. The right method depends on query type, dataset, and latency requirement. BM25 is strongest for precise keyword-based search, especially with rare terms, proper nouns, and technical vocabulary. BERT-based dense retrieval is better for semantic search where query and document wording may differ.

HNSW is useful when fast vector retrieval is needed. In the main results, BERT + HNSW preserves the retrieval quality of BERT + FAISS while reducing query latency. Cross-encoder reranking is useful when precision matters more than latency, making it suitable as a second-stage component.

Overall, the project successfully demonstrates a complete modern retrieval pipeline including preprocessing, embedding generation, BM25 search, dense vector retrieval, approximate nearest-neighbour indexing, evaluation, and Streamlit deployment.

Acknowledgments

We thank Prof. Anirban Dasgupta and the course teaching team for their guidance and support throughout this project.

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*. 4171–4186.

[2] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[3] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[4] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and Their Compositionality. In *Proceedings of NeurIPS*. 3111–3119.

[5] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. In *Proceedings of the NeurIPS Workshop*.
[6] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP*. 3982–3992.
[7] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.
[8] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of NeurIPS Datasets and Benchmarks*.